Documentación de Cambios, Modificaciones y Nuevas Características

Proyecto: Compilador Lenguaje ZAP (`AnalizadorLexico_LenguajeZAP`)
Fecha: 29 de Octubre de 2026
Rama: `main`
Archivo principal modificado: `AnalizadorLexico_LenguajeZAP/Form1.cs`
Archivos inspeccionados: `AnalizadorLexico_LenguajeZAP/PushdownParser.cs`

Índice

1. Verificación de Sentencia Switch-Case con Múltiples 'Case'
2. Detección de Variables Duplicadas (Redeclaración de Variables)
3. Detección y Mensajes Explícitos de Desbordamiento Numérico (Overflow)
4. Control de Zoom Dinámico en el Editor de Código
5. Soporte de Asignaciones de Variables dentro de Bloques 'Case'
6. Resumen de Pruebas Automatizadas de Validación

---

1. Verificación de Sentencia Switch-Case con Múltiples 'Case'

Razón

Se requería verificar que la sentencia `switch-case` admitiese más de un `case` de manera consecutiva y evaluar que la gramática del compilador y el análisis semántico no impidieran múltiples casos.

Inspección en la Gramática (`PushdownParser.cs`)

En la definición formal del autómata de pila (`PushdownParser.cs`):

```
// Gramática de casos en el parser sintáctico:
// Regla 30: SW -> PR30 ARGSW CS14 BODY PR31 CS15 SW   (case <valor> : <body> break; <SW>)
// Regla 31: SW -> PR32 CS14 BODY PR31 CS15           (default : <body> break;)
// Regla 32: SW -> ε                                   (cadena vacía)
```

Explicación

- El símbolo no terminal `SW` es **recursivo por la derecha**. Cada cláusula `case` (`PR30`) consume su argumento, dos puntos (`CS14`), cuerpo de instrucciones (`BODY`), `break` (`PR31`) y punto y coma (`CS15`), invocando nuevamente a `SW` .
- Esto garantiza gramaticalmente la admisión de **1, 2, 3 o N casos**, seguidos opcionalmente por un `default` (`PR32`).
- Además, en el análisis semántico de `Form1.cs` ya existía validación para impedir valores duplicados en los distintos `case` del mismo switch.

---

2. Detección de Variables Duplicadas (Redeclaración de Variables)

Razón

Anteriormente, al declarar una variable con un nombre que ya había sido declarado (ya sea en el mismo bloque o en un bloque padre), el compilador no emitía ningún error semántico; sobrescribía silenciosamente los datos en la tabla de símbolos o en la pila de ámbitos. Se requería emitir un error semántico explícito al detectar dos variables con el mismo nombre.

Cambio 2.1: Expansión de Tipos de Datos Reconocidos en Declaraciones

Código Anterior:

```
string lexemaMin = actual.Lexema.ToLower();
bool esDeclaracion = actual.Token == "PR19" || actual.Token == "PR28" || actual.Token == "PR02" || ac
                     lexemaMin == "int" || lexemaMin == "float" || lexemaMin == "string" ||
                     lexemaMin == "char" || lexemaMin == "bool";

bool esVariableEnDeclaracion = false;
if (i > 0)
{
    var anterior = listaTokensCompletos[i - 1];
    string antMin = anterior.Lexema.ToLower();
    if (anterior.Token == "PR19" || anterior.Token == "PR28" || anterior.Token == "PR02" || anterior.
        antMin == "int" || antMin == "float" || antMin == "string" || antMin == "char" || antMin == "
    {
        esVariableEnDeclaracion = true;
    }
}
```

Código Nuevo:

```
string lexemaMin = actual.Lexema.ToLower();
bool esDeclaracion = actual.Token == "PR19" || actual.Token == "PR28" || actual.Token == "PR02" || ac
                     actual.Token == "PR11" || actual.Token == "PR13" || actual.Token == "PR03" || ac
                     lexemaMin == "int" || lexemaMin == "float" || lexemaMin == "double" ||
                     lexemaMin == "string" || lexemaMin == "char" || lexemaMin == "bool" || lexemaMin

bool esVariableEnDeclaracion = false;
if (i > 0)
{
    var anterior = listaTokensCompletos[i - 1];
    string antMin = anterior.Lexema.ToLower();
    if (anterior.Token == "PR19" || anterior.Token == "PR28" || anterior.Token == "PR02" || anterior.
        anterior.Token == "PR11" || anterior.Token == "PR13" || anterior.Token == "PR03" || anterior.
        antMin == "int" || antMin == "float" || antMin == "double" || antMin == "string" || antMin ==
    {
        esVariableEnDeclaracion = true;
    }
}
```

Explicación del Código:

Se agregaron los identificadores y tokens faltantes como `double` (`PR13`), `char` (`PR03` / `PRO3`), `boolean` (`PRO2`) para que cualquier variable declarada con estos tipos sea identificada correctamente durante el análisis semántico.

Cambio 2.2: Validación de Existencia Previa en Ámbito Actual y Ámbitos Exteriores

Código Anterior:

```
if (esDeclaracion && i + 1 < listaTokensCompletos.Count)
{
    var siguiente = listaTokensCompletos[i + 1];
    if (siguiente.Token == "IDEN")
    {
        string idenLexema = siguiente.Lexema;
        string tipoNormalizado = NormalizarTipo(actual.Lexema, actual.Token);

        tablaTiposVariables[siguiente.Token] = tipoNormalizado;
        ambitos.Peek().Add(idenLexema);

        if (tablaSimbolos.ContainsKey(idenLexema))
        {
            Simbolo sim = tablaSimbolos[idenLexema];
            sim.TipoDato = tipoNormalizado;
            sim.Scope = (nivelBloque == 0) ? "Global" : "Local";

            int bytes = ObtenerTamanoBytes(tipoNormalizado);
            sim.Tamano = bytes;
            if (sim.Scope == "Global")
            {
                sim.Direccion = offsetGlobal;
                offsetGlobal += bytes;
            }
            else
            {
                sim.Direccion = offsetLocal;
                offsetLocal += bytes;
            }

            sim.EstaInicializada = true;
        }
    }
}
```

Código Nuevo:

```
if (esDeclaracion && i + 1 < listaTokensCompletos.Count)
{
    var siguiente = listaTokensCompletos[i + 1];
    if (siguiente.Token == "IDEN")
    {
        string idenLexema = siguiente.Lexema;
        string tipoNormalizado = NormalizarTipo(actual.Lexema, actual.Token);

        // Detección de error por variables con el mismo nombre (redeclaración)
        bool yaDeclaradaMismoAmbito = ambitos.Peek().Contains(idenLexema);
        bool yaDeclaradaExterior = false;
        foreach (var scope in ambitos)
        {
            if (scope.Contains(idenLexema))
            {
                yaDeclaradaExterior = true;
                break;
            }
        }

        if (yaDeclaradaMismoAmbito)
        {
            ImprimirErrorSemantico(siguiente.Linea, $"Error Semántico: La variable '{idenLexema}' ya
            erroresSemanticos++;
        }
        else if (yaDeclaradaExterior)
        {
            ImprimirErrorSemantico(siguiente.Linea, $"Error Semántico: La variable '{idenLexema}' ya
            erroresSemanticos++;
        }
        else
        {
            tablaTiposVariables[siguiente.Token] = tipoNormalizado;
            ambitos.Peek().Add(idenLexema);

            if (tablaSimbolos.ContainsKey(idenLexema))
            {
                Simbolo sim = tablaSimbolos[idenLexema];
                sim.TipoDato = tipoNormalizado;
                sim.Scope = (nivelBloque == 0) ? "Global" : "Local";

                int bytes = ObtenerTamanoBytes(tipoNormalizado);
                sim.Tamano = bytes;
                if (sim.Scope == "Global")
                {
                    sim.Direccion = offsetGlobal;
                    offsetGlobal += bytes;
                }
                else
                {
                    sim.Direccion = offsetLocal;
                    offsetLocal += bytes;
                }

                sim.EstaInicializada = true;
            }
        }
    }
}
```

Explicación del Código:

- Se consulta el conjunto hash del tope de la pila de ámbitos: `ambitos.Peek().Contains(idenLexema)`. Si ya existe en el mismo bloque, se reporta: "*La variable '...' ya fue declarada previamente en este mismo ámbito*".
- Se recorren los ámbitos contenedores (`foreach (var scope in ambitos)`). Si ya existe en un ámbito superior o global, se reporta: "*La variable '...' ya fue declarada previamente en un ámbito exterior (no se permite duplicar nombres de variables)*".
- Si la variable está duplicada, se incrementa `erroresSemanticos` y se evita registrar el duplicado en la tabla de símbolos o en el ámbito para prevenir inconsistencias de tipos posteriores.

Cambio 2.3: Normalización de Tipos de Tokens de Palabras Reservadas en `NormalizarTipo`

Código Anterior:

```
private string NormalizarTipo(string lexema, string token)
{
    if (token == "PR19" || lexema == "int") return "int";
    if (token == "PR28" || lexema == "string") return "string";
    if (token == "PR02" || lexema == "boolean" || lexema == "bool") return "boolean";
    if (token == "PR11" || lexema == "float" || lexema == "double") return "float";
    if (lexema == "char") return "char";
    return lexema;
}
```

Código Nuevo:

```
private string NormalizarTipo(string lexema, string token)
{
    if (token == "PR19" || lexema == "int") return "int";
    if (token == "PR28" || lexema == "string") return "string";
    if (token == "PR02" || token == "PRO2" || lexema == "boolean" || lexema == "bool") return "boolea
    if (token == "PR11" || token == "PR13" || lexema == "float" || lexema == "double") return "float"
    if (token == "PR03" || token == "PRO3" || lexema == "char") return "char";
    return lexema;
}
```

Explicación del Código:

Soporta variantes donde el token viene como `PRO2` (con letra O) o `PR02` (con cero), así como `PR13` para `double` y `PR03` / `PRO3` para `char`.

---

3. Detección y Mensajes Explícitos de Desbordamiento Numérico (Overflow)

Razón

1. En el método `ValidarRangoNumerico`, cuando una constante numérica superaba los límites de 32 bits (`int.MinValue` a `int.MaxValue`), el método retornaba `false`, pero dejaba la variable de salida `mensajeError = string.Empty;`. Como resultado, la consola de errores semánticos imprimía únicamente: `• LÍNEA X:`, sin texto descriptivo.
2. No se verificaba desbordamiento en el valor de las etiquetas `case` numéricas.

3. No se verificaba desbordamiento resultante de operaciones aritméticas compuestas (por ejemplo: `int $res = 2000000000 + 2000000000;`).

Cambio 3.1: Generación de Mensaje de Error en `ValidarRangoNumerico`

Código Anterior:

```
public bool ValidarRangoNumerico(string tokenValor, string tipoDato, out string mensajeError)
{
    mensajeError = string.Empty;

    if (tipoDato.Equals("int", StringComparison.OrdinalIgnoreCase))
    {
        if (!long.TryParse(tokenValor, out long valorLong) || valorLong < int.MinValue || valorLong >
        {

            return false;
        }
    }
    else if (tipoDato.Equals("float", StringComparison.OrdinalIgnoreCase) || tipoDato.Equals("double"
    {
        if (!double.TryParse(tokenValor, out double valorDouble) || double.IsInfinity(valorDouble))
        {
            return false;
        }
    }

    return true;
}
```

Código Nuevo:

```
public bool ValidarRangoNumerico(string tokenValor, string tipoDato, out string mensajeError)
{
    mensajeError = string.Empty;

    if (tipoDato.Equals("int", StringComparison.OrdinalIgnoreCase) || tipoDato.Equals("pr19", StringC
    {
        if (!long.TryParse(tokenValor, out long valorLong) || valorLong < int.MinValue || valorLong >
        {
            mensajeError = $"Error de Desbordamiento (Overflow): El valor entero '{tokenValor}' exced
            return false;
        }
    }
    else if (tipoDato.Equals("float", StringComparison.OrdinalIgnoreCase) || tipoDato.Equals("pr11",
    {
        if (!float.TryParse(tokenValor, out float valorFloat) || float.IsInfinity(valorFloat))
        {
            mensajeError = $"Error de Desbordamiento (Overflow): El valor numérico '{tokenValor}' exc
            return false;
        }
    }
    else if (tipoDato.Equals("double", StringComparison.OrdinalIgnoreCase))
    {
        if (!double.TryParse(tokenValor, out double valorDouble) || double.IsInfinity(valorDouble))
        {
            mensajeError = $"Error de Desbordamiento (Overflow): El valor numérico '{tokenValor}' exc
            return false;
        }
    }

    return true;
}
```

Explicación del Código:

- Al ocurrir un desbordamiento en enteros (ya sea que `long.TryParse` falle o el valor esté fuera de `[-2147483648, 2147483647]`), se construye un mensaje claro indicando el lexema causante y el rango permitido.
- Se agregó el manejo de desbordamiento para números flotantes de 32 bits (`float.IsInfinity`) y dobles de 64 bits (`double.IsInfinity`).

Cambio 3.2: Validación de Desbordamiento en Valores de Etiquetas 'Case'

Código Añadido (en el procesamiento de `switch` en `AnalizadorSemantico`):

```
// Determinar el tipo del valor del case actual
string tipoCaseDetectado = "";
if (tokenTipoCase == "CONENTERO" || int.TryParse(valorCase, out _))
{
    tipoCaseDetectado = "int";
    if (tokenTipoCase == "CONENTERO")
    {
        if (!ValidarRangoNumerico(valorCase, "int", out string msgOverflowCase))
        {
            ImprimirErrorSemantico(tokenActualSwitch.Linea, msgOverflowCase);
            erroresSemanticos++;
        }
    }
}
```

Razón y Explicación:

Si el usuario escribe un caso como `case 999999999999:`, anteriormente era etiquetado como `CONENTERO` por el léxico, pero no se validaba si cabía en 32 bits antes de compararlo con la variable del switch. Ahora se ejecuta `ValidarRangoNumerico` y se imprime el mensaje de desbordamiento en la línea correspondiente.

Cambio 3.3: Detección de Desbordamiento en Resultados de Expresiones Evaluadas

Código Añadido (en la validación de asignaciones aritméticas de tipo entero en `AnalizadorSemantico`):

```
// Validación de desbordamiento (Overflow) en el resultado de expresiones compuestas evaluadas
try
{
    string exprParaEvaluar = string.Join(" ", expTokens.Select(x => x.Lexema));
    exprParaEvaluar = Regex.Replace(exprParaEvaluar, @"\$[a-zA-Z0-9_]+", m =>
    {
        if (tablaSimbolos.ContainsKey(m.Value) && tablaSimbolos[m.Value].Valor != null)
        {
            string v = tablaSimbolos[m.Value].Valor.ToString();
            if (double.TryParse(v, out _)) return v;
        }
        return m.Value;
    });

    if (Regex.IsMatch(exprParaEvaluar, @"^[0-9\.\s\+\-\*\/\(\)]+$"))
    {
        DataTable dtCalc = new DataTable();
        var calcResult = dtCalc.Compute(exprParaEvaluar, "");
        if (calcResult != null && double.TryParse(calcResult.ToString(), out double resNum))
        {
            if (resNum < int.MinValue || resNum > int.MaxValue)
            {
                ImprimirErrorSemantico(actual.Linea, $"Error de Desbordamiento (Overflow): El resulta
                erroresSemanticos++;
            }
        }
    }
}
catch { }
```

Razón y Explicación:

Permite detectar desbordamiento en tiempo de compilación cuando la expresión aritmética contiene constantes o variables con valor conocido cuya operación acumulada excede el rango de un entero de 32 bits.

---

4. Control de Zoom Dinámico en el Editor de Código

Razón

Se requería permitir al usuario cambiar el tamaño de la letra del código en la ventana del editor de texto (funcionalidad de Zoom), tanto visualmente mediante controles en la interfaz como a través de atajos de teclado y la rueda del ratón (`Ctrl + Scroll`).

Cambio 4.1: Atributos y Constantes de Zoom

Código Añadido (en la cabecera de la clase `Form1`):

```
// Variables y controles para zoom dinámico en el editor de código
private float _tamanoFuenteCodigo = 10.5f;
private const float TAMANO_MIN_FUENTE = 7.0f;
private const float TAMANO_MAX_FUENTE = 36.0f;
private const float TAMANO_DEFAULT_FUENTE = 10.5f;

private Panel panelZoomControls;
private Label lblZoomLevel;
private Button btnZoomIn;
private Button btnZoomOut;
private Button btnZoomReset;
```

Explicación del Código:

- Establece los límites mínimos (7pt) y máximos (36pt) de fuente para evitar que el texto se vuelva ilegible o desborde desproporcionadamente la interfaz.
- Declara las instancias de controles que conformarán la barra interactiva de zoom.

Cambio 4.2: Métodos de Configuración y Lógica de Zoom

Código Añadido (en una región dedicada en `Form1`):

```
#region Configuración de Zoom del Editor de Código

private void ConfigurarZoom()
{
    if (rTxtCodigoFuente != null)
    {
        _tamanoFuenteCodigo = rTxtCodigoFuente.Font.Size;
        rTxtCodigoFuente.MouseWheel += RTxtCodigoFuente_MouseWheel;
    }
    if (rTxtNumeros != null)
    {
        rTxtNumeros.MouseWheel += RTxtCodigoFuente_MouseWheel;
    }

    CrearControlesVisualesZoom();
}

private void CrearControlesVisualesZoom()
{
    if (panelCodigoHeader == null) return;

    panelZoomControls = new Panel
    {
        Dock = DockStyle.Right,
        Width = 205,
        Height = 34,
        BackColor = Color.Transparent
    };

    Label lblZoomTitulo = new Label
    {
        Text = "Zoom:",
        ForeColor = Color.FromArgb(148, 163, 184),
        Font = new Font("Segoe UI", 8.25F, FontStyle.Regular),
        AutoSize = false,
        Width = 44,
        Height = 26,
        Top = 4,
        Left = 0,
        TextAlign = ContentAlignment.MiddleRight
    };

    btnZoomOut = new Button
    {
        Text = "-",
        Font = new Font("Segoe UI", 9.5F, FontStyle.Bold),
        ForeColor = Color.White,
        BackColor = Color.FromArgb(51, 65, 85),
        FlatStyle = FlatStyle.Flat,
        Width = 28,
        Height = 24,
        Top = 5,
        Left = 48,
        Cursor = Cursors.Hand
    };
    btnZoomOut.FlatAppearance.BorderSize = 0;
    btnZoomOut.Click += (s, e) => CambiarZoomCodigo(-1.0f);

    lblZoomLevel = new Label
    {
        Text = "100%",
        Font = new Font("Segoe UI Semibold", 8.5F),
```

```
            ForeColor = Color.FromArgb(226, 232, 240),
            BackColor = Color.FromArgb(30, 41, 59),
            BorderStyle = BorderStyle.FixedSingle,
            AutoSize = false,
            Width = 50,
            Height = 24,
            Top = 5,
            Left = 80,
            TextAlign = ContentAlignment.MiddleCenter,
            Cursor = Cursors.Hand
        };
        lblZoomLevel.Click += (s, e) => RestablecerZoomCodigo();

        btnZoomIn = new Button
        {
            Text = "+",
            Font = new Font("Segoe UI", 9.5F, FontStyle.Bold),
            ForeColor = Color.White,
            BackColor = Color.FromArgb(51, 65, 85),
            FlatStyle = FlatStyle.Flat,
            Width = 28,
            Height = 24,
            Top = 5,
            Left = 134,
            Cursor = Cursors.Hand
        };
        btnZoomIn.FlatAppearance.BorderSize = 0;
        btnZoomIn.Click += (s, e) => CambiarZoomCodigo(1.0f);

        btnZoomReset = new Button
        {
            Text = "↺",
            Font = new Font("Segoe UI", 8.5F, FontStyle.Bold),
            ForeColor = Color.FromArgb(148, 163, 184),
            BackColor = Color.FromArgb(51, 65, 85),
            FlatStyle = FlatStyle.Flat,
            Width = 24,
            Height = 24,
            Top = 5,
            Left = 166,
            Cursor = Cursors.Hand
        };
        btnZoomReset.FlatAppearance.BorderSize = 0;
        btnZoomReset.Click += (s, e) => RestablecerZoomCodigo();

        ToolTip tip = new ToolTip();
        tip.SetToolTip(btnZoomOut, "Disminuir tamaño de letra (Ctrl - o Ctrl + Rueda abajo)");
        tip.SetToolTip(lblZoomLevel, "Restablecer al 100% (Ctrl 0)");
        tip.SetToolTip(btnZoomIn, "Aumentar tamaño de letra (Ctrl + o Ctrl + Rueda arriba)");
        tip.SetToolTip(btnZoomReset, "Restablecer tamaño predeterminado (Ctrl 0)");

        panelZoomControls.Controls.Add(lblZoomTitulo);
        panelZoomControls.Controls.Add(btnZoomOut);
        panelZoomControls.Controls.Add(lblZoomLevel);
        panelZoomControls.Controls.Add(btnZoomIn);
        panelZoomControls.Controls.Add(btnZoomReset);

        if (lblCodigoHeaderTip != null)
        {
            lblCodigoHeaderTip.Text = "✦ Ctrl+V: pegar | F5: Compilar | Zoom: Ctrl+Rueda o Ctrl+/-";
        }

        panelCodigoHeader.Controls.Add(panelZoomControls);
        panelZoomControls.BringToFront();
    }

    public void CambiarZoomCodigo(float delta)
```

```
{
    float nuevoTamano = _tamanoFuenteCodigo + delta;
    if (nuevoTamano < TAMANO_MIN_FUENTE) nuevoTamano = TAMANO_MIN_FUENTE;
    if (nuevoTamano > TAMANO_MAX_FUENTE) nuevoTamano = TAMANO_MAX_FUENTE;

    AplicarTamanoFuenteCodigo(nuevoTamano);
}

public void RestablecerZoomCodigo()
{
    AplicarTamanoFuenteCodigo(TAMANO_DEFAULT_FUENTE);
}

public void AplicarTamanoFuenteCodigo(float nuevoTamano)
{
    _tamanoFuenteCodigo = (float)Math.Round(nuevoTamano, 1);
    Font nuevaFuente = new Font("Consolas", _tamanoFuenteCodigo, FontStyle.Regular);

    rTxtCodigoFuente.Font = nuevaFuente;
    rTxtNumeros.Font = nuevaFuente;

    Size size = TextRenderer.MeasureText("9999", nuevaFuente);
    int anchoDeseado = Math.Max(52, size.Width + 14);
    rTxtNumeros.Width = anchoDeseado;

    if (this.IsHandleCreated)
    {
        ActualizarNumerosLinea(null, null);
    }

    if (lblZoomLevel != null)
    {
        int porcentaje = (int)Math.Round((_tamanoFuenteCodigo / TAMANO_DEFAULT_FUENTE) * 100);
        lblZoomLevel.Text = $"{porcentaje}%";
    }
}

private void RTxtCodigoFuente_MouseWheel(object sender, MouseEventArgs e)
{
    if ((Control.ModifierKeys & Keys.Control) == Keys.Control)
    {
        if (e.Delta > 0)
        {
            CambiarZoomCodigo(1.0f);
        }
        else if (e.Delta < 0)
        {
            CambiarZoomCodigo(-1.0f);
        }
        ((HandledMouseEventArgs)e).Handled = true;
    }
}

#endregion
```

Explicación del Código:

- `ConfigurarZoom()` se invoca al final del evento `Form1_Load`.
- `CrearControlesVisualesZoom()` inyecta en el encabezado del editor (`panelCodigoHeader`) los botones `-`, `+`, el indicador interactivo `%` y el botón de reinicio `↺` con estilos acordes al tema oscuro.
- `AplicarTamanoFuenteCodigo()` cambia la fuente tanto de `rTxtCodigoFuente` como de la barra lateral de numeración `rTxtNumeros`, recalculando automáticamente el ancho necesario de la columna de números con `TextRenderer.MeasureText` para evitar desalineación entre las líneas de código y sus números.

- `RTxtCodigoFuente_MouseWheel()` intercepta el evento de scroll con la tecla `Ctrl` presionada para aumentar o disminuir el tamaño dinámicamente como en editores profesionales (VS Code, Visual Studio).

Cambio 4.3: Soporte de Atajos de Teclado en `ProcessCmdKey`

Código Anterior:

```
protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    if (keyData == Keys.F5)
    {
        btnAnalizarCodigo_Click(btnAnalizarCodigo, EventArgs.Empty);
        return true;
    }
    return base.ProcessCmdKey(ref msg, keyData);
}
```

Código Nuevo:

```
protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    if (keyData == Keys.F5)
    {
        btnAnalizarCodigo_Click(btnAnalizarCodigo, EventArgs.Empty);
        return true;
    }
    if (keyData == (Keys.Control | Keys.Oemplus) || keyData == (Keys.Control | Keys.Add) || keyData =
    {
        CambiarZoomCodigo(1.0f);
        return true;
    }
    if (keyData == (Keys.Control | Keys.OemMinus) || keyData == (Keys.Control | Keys.Subtract))
    {
        CambiarZoomCodigo(-1.0f);
        return true;
    }
    if (keyData == (Keys.Control | Keys.D0) || keyData == (Keys.Control | Keys.NumPad0))
    {
        RestablecerZoomCodigo();
        return true;
    }
    return base.ProcessCmdKey(ref msg, keyData);
}
```

Explicación del Código:

Se añaden los atajos globales para la ventana:
- `Ctrl + '+'` (tanto en teclado principal como en el numérico): Aumenta el zoom en 1pt.
- `Ctrl + '-'` (teclado principal y numérico): Reduce el zoom en 1pt.
- `Ctrl + '0'` (teclado principal y numérico): Restablece el zoom al 100% predeterminado (10.5pt).

Cambio 4.4: Adaptación del Tamaño de Fuente en `PegarTextoPlano`

Código Anterior:

```
rTxtCodigoFuente.SelectedText = texto;
rTxtCodigoFuente.Select(inicio, texto.Length);
rTxtCodigoFuente.SelectionFont = new Font("Consolas", 10.2f);
rTxtCodigoFuente.SelectionStart = inicio + texto.Length;
rTxtCodigoFuente.SelectionLength = 0;
```

Código Nuevo:

```
rTxtCodigoFuente.SelectedText = texto;
rTxtCodigoFuente.Select(inicio, texto.Length);
rTxtCodigoFuente.SelectionFont = new Font("Consolas", _tamanoFuenteCodigo);
rTxtCodigoFuente.SelectionStart = inicio + texto.Length;
rTxtCodigoFuente.SelectionLength = 0;
```

Razón y Explicación:

Anteriormente, al pegar texto sin formato (`Ctrl + V`), el editor forzaba la selección al tamaño fijo `10.2f` . Al cambiar a `_tamanoFuenteCodigo` , el texto pegado respeta fielmente el nivel de zoom activo en ese momento.

---

5. Soporte de Asignaciones de Variables dentro de Bloques 'Case'

Razón

Durante las pruebas de múltiples casos en la sentencia `switch-case` , se detectó que el generador de código intermedio (`ProcesarBloqueDinamico`) y la máquina virtual P (`ProcesarBloqueTokens`) únicamente parseaban sentencias `print(...)` dentro de los bloques de cada `case` . Si el usuario colocaba una asignación dentro de un caso (por ejemplo: `case 1: $x = 10; break;`), la asignación no se registraba en las cuádruplas, tríplos ni código P.

Cambio 5.1: En `ProcesarBloqueTokens` (Generación de Código P)

Código Añadido:

```
else if (tokens[index].Lexema == "=" || tokens[index].Token == "OPASIG" || (index + 1 < tokens.Count
{
    int idxAsig = (tokens[index].Lexema == "=" || tokens[index].Token == "OPASIG") ? index : index +
    string varName = tokens[idxAsig - 1].Lexema;

    List<string> asigTokens = new List<string>();
    index = idxAsig + 1;
    while (index < tokens.Count && tokens[index].Lexema != ";" && tokens[index].Lexema != "}")
    {
        asigTokens.Add(tokens[index].Lexema);
        index++;
    }
    if (index < tokens.Count && tokens[index].Lexema == ";") index++;

    if (asigTokens.Count > 0 && casoActual != null)
    {
        var instsAsig = InstruccionP.GenerarCodigoDinamico(varName, asigTokens);
        casoActual.Instrucciones.AddRange(instsAsig);
    }
}
```

Explicación del Código:

Detecta la asignación a una variable dentro del cuerpo del `case` actual y compila las instrucciones dinámicas de la Máquina P asociadas al caso correspondiente.

Cambio 5.2: En `ProcesarBloqueDinamico` (Generación de Cuádruplos, Tríplos y TAC)

Código Añadido:

```
else if (tokens[index].Lexema == "=" || tokens[index].Token == "OPASIG" || (index + 1 < tokens.Count
{
    int idxAsig = (tokens[index].Lexema == "=" || tokens[index].Token == "OPASIG") ? index : index +
    string varName = tokens[idxAsig - 1].Lexema;

    List<string> expToks = new List<string>();
    index = idxAsig + 1;
    while (index < tokens.Count && tokens[index].Lexema != ";" && tokens[index].Lexema != ",")
    {
        expToks.Add(tokens[index].Lexema);
        index++;
    }
    if (index < tokens.Count && tokens[index].Lexema == ";") index++;

    if (expToks.Count > 0)
    {
        var resComp = CompiladorExpresiones.Compilar(expToks, cuadruplos, triplos, tac);
        cuadruplos.Add(new Cuadruplo("=", resComp.ResultadoCuadruplo, "-", varName));
        triplos.Add(new Triplo("=", varName, resComp.ResultadoTriplo));
        tac.Add($"{varName} = {resComp.ResultadoCuadruplo}");
    }
}
```

Explicación del Código:

Genera la representación de tres direcciones (TAC), las cuádruplas y los tríplos correspondientes a expresiones evaluadas dentro de los casos del switch.

---

6. Resumen de Pruebas Automatizadas de Validación

Se ejecutó una suite exhaustiva de 17 pruebas unitarias y de integración que validaron cada uno de los requisitos:

#	Prueba	Caso de Entrada	Resultado Esperado	Estado
1	Múltiples Casos Switch (3 casos válidos)	`switch($opcion) { case 1: ... case 2: ... case 3: ... }`	0 Errores sintácticos / semánticos	**PASADA**
2	Múltiples Casos Switch (Duplicado)	`switch($opcion) { case 1: ... case 1: ... }`	Error Semántico: Case duplicado	**PASADA**
3	Múltiples Casos Switch (Incompatibilidad tipo)	`switch($opcion) { case "cadena": ... }` siendo `$opcion` int	Error Semántico: Incompatibilidad de tipos	**PASADA**
4	Redeclaración mismo ámbito	`int $x; int $x;`	Error: La variable ya fue declarada en este mismo ámbito	**PASADA**
5	Redeclaración ámbito anidado	`int $x; if ($x == 0) { int $x; }`	Error: La variable ya fue declarada en un ámbito exterior	**PASADA**
6	Variables con nombres distintos	`int $x; int $y; string $z;`	0 Errores	**PASADA**
7	Overflow entero positivo	`int $grande = 999999999999;`	Error de Desbordamiento (Overflow): excede 32 bits	**PASADA**
8	Overflow entero negativo	`int $negativo = -999999999999;`	Error de Desbordamiento (Overflow): excede 32 bits	**PASADA**
9	Límite máximo válido `int.MaxValue`	`int $limite = 2147483647;`	0 Errores de overflow	**PASADA**
10	Límite mínimo válido `int.MinValue`	`int $limite = -2147483648;`	0 Errores de overflow	**PASADA**
11	Overflow en etiqueta `case`	`switch($x) { case 999999999999: ... }`	Error de Desbordamiento (Overflow) en case	**PASADA**
12	Overflow en expresión evaluada	`int $resultado = 2000000000 + 2000000000;`	Error de Desbordamiento (Overflow): excede límites	**PASADA**
13	Zoom aumentar fuente	`CambiarZoomCodigo(+1.0f)`	Fuente incrementada a 11.5pt	**PASADA**
14	Zoom disminuir fuente	`CambiarZoomCodigo(-1.0f)`	Fuente disminuida a 9.5pt	**PASADA**
15	Zoom límite superior	`CambiarZoomCodigo(+50.0f)`	Limitado a `TAMANO_MAX_FUENTE` (36pt)	**PASADA**
16	Zoom límite inferior	`CambiarZoomCodigo(-50.0f)`	Limitado a `TAMANO_MIN_FUENTE` (7pt)	**PASADA**
17	Zoom restablecer	`RestablecerZoomCodigo()`	Restablecido a `TAMANO_DEFAULT_FUENTE` (10.5pt)	**PASADA**

Resultado Global: `17 PASADAS, 0 FALLIDAS`.
Compilación final con MSBuild: `0 Advertencias, 0 Errores`.